Sonoscopio: technical project report

Real-time audio analyzer for the browser · Portfolio report

Versione italiana

Table of contents

- 1. Introduction and goals
- 2. Features
- 3. Tools and technologies
- 4. Project structure
- 5. Core concepts
- 6. Architecture and workflow
- 7. Code walkthrough, section by section
- 8. Design decisions and rationale
- 9. Project evolution and problems solved
- 10. Verification and testing
- 11. Known limitations
- 12. Possible future work
- 13. Running and publishing
- Appendix A: Mapping between original and renamed identifiers
- Appendix B: Glossary

> **Note on language.** The user interface and the names used inside the code (files, variables and functions) are in Italian, by design. Every identifier is explained the first time it appears, and Appendix A maps them to the original English names.

1. Introduction and goals

Sonoscopio is a web application that listens to a sound, coming from an audio file or from the microphone, breaks it down into its frequencies and displays it in real time in three different visualizations. Next to the graphs it shows an overall level meter with RMS and peak values, inspired by the tools used in mastering studios.

The project has three goals:

- to build a portfolio piece that opens with a single click, with nothing to install, and that demonstrates skills in digital audio and front-end development;
- to apply and consolidate the knowledge gained during university studies (JavaScript, signal processing, multimedia computing, creative computing);
- to understand every part of the code thoroughly: for this reason the project was built in small steps and commented in detail.

The project uses no libraries or frameworks: only plain HTML, CSS and JavaScript, together with the browser APIs. This is a deliberate choice, because it shows the ability to work with what lies «under the hood» of libraries.

2. Features

- **Two audio sources**: a file from the computer (MP3, WAV, OGG...) or the microphone, with a button that switches it on and off.
- **Three visualization modes**: Bars, Spectrum (a frequency/level curve) and Spectrogram (time, frequency and intensity shown as color). In the interface they are labeled *Barre*, *Spettro* and *Sonogramma*.
- **Graduated Cartesian axes** in every mode. Bars and Spectrum show frequency in Hz on the horizontal axis (logarithmic scale from 20 Hz to 20 kHz) and level in dB on the vertical axis. The Spectrogram shows time on the horizontal axis and frequency on the vertical axis.
- **Color scale and level meter** in the Spectrogram.
- **RMS and peak box**, visible in every mode, with calculations spread over time rather than instantaneous.
- **Smoothness slider**, which controls how soft the movement of bars and spectrum is.
- **Error handling**: if microphone permission is denied, a message explains what to do.
- **Careful interface**: consistent palette, keyboard navigation, labels associated with the controls.

3. Tools and technologies

Technology	Role in the project
HTML5	Page structure: canvas, buttons, menu, slider
CSS3	Appearance: colors, layout with flexbox, adaptation to narrow screens
	Application logic: constants, functions, loops, events,

JavaScript (ES6+)	asynchronous functions
Web Audio API	Building the audio chain and analyzing the sound (`AudioContext`, `AnalyserNode`, sources, gain node)
Canvas 2D API	Drawing the graphs pixel by pixel
`requestAnimationFrame`	Animation loop synchronized with the screen
`getUserMedia`	Requesting access to the microphone
Sublime Text	Editor used to write the code
Node.js	Used only for automated checks during development; it is not needed to run the project
Git and GitHub	Publishing and version control

4. Project structure

```
sonoscopio/
├── index.html          page structure
├── css/
│   └── stile.css       appearance
├── js/
│   └── sonoscopio.js   behavior
└── docs/
    ├── RELAZIONE.it.md report in Italian
    └── REPORT.en.md    report in English
```

The code is split into three files, each with a single job: the HTML says **what is** on the page, the CSS says **what it looks like**, and the JavaScript says **what it does**. This separation of concerns is good practice: to change a color you open the CSS, to change a behavior you open the JavaScript, without searching through one long file.

The main file is called `index.html` because that is the name web servers, GitHub Pages included, look for first. Renaming it would prevent the page from opening when only the folder address is typed.

The JavaScript lives in a single file divided into nine numbered sections (described in chapter 7). I deliberately avoided ES modules (`import` and `export`), because they require a local server even just to try the project. With a regular `<script src="...">` tag, a double click on `index.html` is enough.

5. Core concepts

5.1 Sound as numbers

To a computer, sound is a long list of numbers, called **samples**, that indicate the height of the sound wave at each instant. A sound card usually produces 44,100 or 48,000 of them per second (the **sampling rate**). Plotting this list gives the **waveform**, the wavy line seen in audio editing programs.

5.2 Spectrum and FFT

The waveform shows how loud the sound is, but not whether it is low or high pitched. The **FFT** (Fast Fourier Transform) breaks the sound down into the frequencies that compose it and tells how much energy each one carries. The result is the **spectrum**: bass on the left, treble on the right. We do not write this calculation ourselves; the browser performs it through the `AnalyserNode`.

5.3 Frequency bins

The analyzer divides the spectrum into «bins»: each one gives us a number from 0 to 255. With `fftSize` equal to 4096 there are 2048 bins; at 44,100 Hz each one is about 10.8 Hz wide. A larger `fftSize` gives more frequency detail but a slower response, because each analysis covers a longer time window (about 93 ms with 4096 samples). This is a classic trade-off, and 4096 turned out to suit the Spectrogram, where the bass frequencies need to be told apart clearly.

5.4 Logarithmic frequency scale

The ear perceives frequencies logarithmically: from 100 to 200 Hz there is a whole octave, while from 10,000 to 10,100 Hz the difference is barely noticeable. With a linear scale almost all the space would go to the treble, where little happens. For this reason the project uses a **logarithmic** scale, in which every octave takes the same amount of space. It is the scale used by professional instruments and it matches the grid lines that are drawn.

5.5 Decibels

The decibel (dB) is a logarithmic scale for sound levels: `20 · log10(amplitude)`. In digital audio, 0 dB is the maximum possible level (**dBFS**, decibels relative to full scale) and every other level is negative. The project shows the spectrum between -100 dB (silence) and -30 dB (top of the graph), and the level meter between -60 and 0 dB.

5.6 RMS and peak

The **RMS** (*root mean square*) measures the average level, and it is closer to what the ear perceives as «loudness». It is computed by squaring the samples, taking their mean and then extracting the square root. The **peak** is instead the highest sample in absolute value, and it shows how close the signal gets to the limit beyond which it distorts.

5.7 The spectrogram

The spectrogram is a three-dimensional graph: **time** runs along the horizontal axis, **frequency** sits on the vertical axis and **intensity** is represented by color. It shows how the composition of the sound changes over time: a sustained note appears as a horizontal line, a percussive hit as a vertical column. In the code and in the interface it is called *sonogramma*.

6. Architecture and workflow

6.1 The audio chain

The Web Audio API works like a studio with instruments connected by cables. The project builds this chain only once:

```
FILE:        lettore -> sorgenteFile -+-> analizzatore -> silenziatore (volume 0)
                                      +-> speakers
MICROPHONE:  microphone ---------------------> analizzatore -> silenziatore (volu
```

The file is also connected to the speakers, because it has to be audible. The microphone, on the other hand, is **not** sent to the speakers: otherwise the sound would come out, re-enter the microphone and create the howl of feedback (the Larsen effect). The analyzer measures the signal without modifying it.

6.2 The operating cycle

1. The user chooses a file or switches the microphone on. The click is mandatory: browsers forbid starting audio without a user action.
2. On first use, the «mixer» (`AudioContext`), the analyzer and the connections are created.
3. At every frame (about 60 times per second) the `disegna` function asks the analyzer for the spectrum and the waveform, filling two lists of numbers.
4. The level measurements (RMS and peak) are updated from the waveform.
5. The canvas is cleared and the mode chosen in the menu is drawn.
6. The box with the level values is drawn.
7. The function asks the browser to call it again at the next frame, and the cycle starts

over.

7. Code walkthrough, section by section

The file `sonoscopio.js` is organized in nine sections. Every function has a name that starts with a verb (`disegnaBarre` , `caricaFileAudio`) because it does something, while constants are written in UPPERCASE. As noted above, the names are in Italian: for example `tela` means canvas, `pennello` means brush and `analizzatore` means analyzer.

Section 1: references to page elements

`document.getElementById` is used once to look up the HTML elements and store them in constants (`tela` , `selettoreFile` , `pulsanteMicrofono` ...). The 2D drawing context of the canvas, called `pennello` , is the object through which every drawing instruction passes.

Section 2: settings

All the adjustable numbers are in one place: FFT size, smoothness, decibel range, number of bars, dimensions and margins of the graphs, frequency and time scales, timings of the level measurements, colors. To change the look or the behavior it is enough to edit one line, without searching through the rest of the code.

Section 3: program state

It holds the variables that change while the program runs, declared with `let` : the mixer and the analyzer (which are `null` , meaning «nothing», at the beginning), the source in use, the current smoothness and the values of the level measurements. This is also where the lists that the analyzer fills with results are created (`datiFrequenza` , a `Uint8Array` , and `datiOnda` , a `Float32Array`), together with the **hidden canvas** of the spectrogram, described later.

Section 4: audio setup

The `preparaAudio` function creates the mixer, the analyzer and the connections described in chapter 6. It begins with an early return: if the mixer already exists, it does nothing. This is mandatory, because the browser allows an `<audio>` element to be connected to the mixer only once.

Section 5: helper functions

These are the «small tools» used by the drawing modes:

- `coloreBarra` produces a color from violet to coral depending on position, varying only

the hue of the HSL model.

- `valoreBanda` returns a value from 0 to 1 for a frequency band. The bands are logarithmic. When a band is narrower than one bin (in the bass) the two neighboring bins are interpolated; when it contains several bins (in the treble) their average is taken.
- `tracciaCurva` joins a list of points with a smooth curve, using quadratic curves and midpoints so that corners disappear.
- `coloreCalore` and `costruisciColoriCalore` create the «magma» palette of the spectrogram by blending five colors. The 256 possible colors are computed only once, because recomputing them for every pixel would be much slower.
- `costruisciTabellaRighe`, `frequenzaAY`, `frequenzaAX` and `dbAY` convert frequencies and decibels into screen positions and back.
- `aggiornaLivelli` computes RMS and peak (see chapter 8).
- `impostaStileTesto`, `disegnaRiquadroLivelli` and `disegnaAssiSpettro` take care of text, the values box and the grid of the axes.

Section 6: the three drawing modes

- **Bars** (`disegnaBarre`): draws the axes first, then one bar for each band. The height is the band value multiplied by the height of the graph. Because the vertical axis of a canvas grows downward, each bar starts from the bottom and rises.
- **Spectrum** (`disegnaCurva`): computes one point per band and joins them with the smooth curve, in two passes. The first fills the area under the curve with a vertical gradient, the second draws the line with a horizontal gradient.
- **Spectrogram** (`disegnaSonogramma` and related functions): uses a second, hidden canvas as memory. At every frame this canvas is copied onto itself shifted to the left, the new columns are drawn on the right edge, and then the canvas is copied onto the visible one, with axes, color scale and meter drawn on top.

Section 7: animation loop

The `disegna` function computes the time elapsed since the previous frame (capped at 0.1 seconds, to avoid jumps if the tab stays in the background), reads the data from the analyzer, updates the levels, clears the canvas, calls the chosen mode and schedules itself again with `requestAnimationFrame`.

Section 8: handling sources and controls

It gathers the functions that react to the user: `caricaFileAudio`, `avviaMicrofono`, `fermaMicrofono`, `commutaMicrofono` and `alCambioFluidita`, plus two utility functions, `mostraMessaggio` and `staSuonando`. `avviaMicrofono` is asynchronous: it uses `await` to

wait for the user's answer to the permission request and a `try/catch` block to handle the case in which it is denied.

Section 9: startup

It sets the slider to its initial value, connects each control to its own function with `addEventListener` and starts the first frame. The function name is passed without parentheses, because the function must not run immediately but only when the event occurs.

8. Design decisions and rationale

- **No libraries.** Plain JavaScript shows an understanding of how Web Audio and Canvas work. The project stays lightweight and opens with a double click.
- **Logarithmic bands and truthful axes.** Bars, Spectrum and Spectrogram share the same frequency scale, so the labels on the axes really correspond to what is drawn.
- **Hidden canvas for the spectrogram.** Redrawing the whole history of the last few seconds every time would be heavy; shifting an image that is already drawn and adding only the new columns is far more efficient.
- **Scrolling tied to time, not to frames.** If one column were always added per frame, the graph would move faster on a 120 Hz screen than on a 60 Hz one. Instead, the elapsed seconds are multiplied by the pixels per second, and leftover fractions of a pixel are accumulated into the next frame.
- **Interpolation in the bass.** In the bass range the pixel rows of the spectrogram outnumber the available bins, and without interpolation visible steps would appear. Each row takes a weighted average between the two neighboring bins.
- **RMS with memory, computed on energy.** An instantaneous value would jump constantly and be unreadable. The project uses an exponential moving average with a time constant of 0.6 seconds: at every frame the value moves by a fraction toward the new measurement, and that fraction depends on the elapsed time. The average is taken on energy (the square of the signal) and only afterwards converted to dB, because averaging dB values directly would be wrong: an instant of silence is minus infinity.
- **Peak hold.** The peak rises immediately, stays still for 1.5 seconds and then falls by 12 dB per second, as a function of time and not of frames.
- **Numbers updated at intervals.** The figures written on screen change four times per second, otherwise they would be unreadable. Bars and curves, instead, stay fluid.
- **Microphone with filters disabled.** Echo cancellation, noise suppression and automatic gain control are designed for video calls and would alter the measurement, so they are turned off.

- **Zero-volume node.** The analyzer is connected to the speakers through a volume control set to zero. Nothing is heard, but this prevents some browsers from stopping the processing of a node that has no output.
- **Separate smoothness for the spectrogram.** The spectrogram uses a fixed, low smoothness, because strong averaging over time would blur rhythmic details.
- **Precomputed color table.** The 256 possible colors are computed once at startup: every column of the spectrogram needs 430 colors, 60 times per second.

9. Project evolution and problems solved

Problem	Cause	Solution
The «Wave» mode was too fast to be understood	The waveform shows only the last few milliseconds of audio and changes completely at every frame	Replaced with a spectrum graph (frequency and level), slower and readable
The «Radial» mode had an empty hole in the center	The lines started from a fixed inner radius	First turned into filled wedges, then replaced by the spectrogram on request
The labels on the axes were not reliable	Bars and spectrum used an «invented» frequency scale	All modes now use the same logarithmic bands as the grid
RMS and peak were too instantaneous	The value of a single frame was used	Exponential moving average, peak hold and numbers updated at intervals
Risk of howling with the microphone	The analyzer output went to the speakers	The microphone is connected only to the analyzer, never to the speakers
Spectrogram speed depended on the screen	One column per frame	Scrolling computed from the elapsed time
No audio during one test	A local configuration error, not a code error	Found with the browser console and isolation tests: a good debugging exercise

10. Verification and testing

The project was verified in three ways:

1. **Syntax check** of every JavaScript file with Node.js.
2. **Automated simulation** with a simulated browser and simulated audio, run on the version before renaming: 15 checks covering switching the microphone on and off, audio connections, the slider, switching sources, handling a denied permission, and the absence of errors during the drawing cycles in all three modes.
3. **Instruction-by-instruction comparison** between the original version and the renamed copy, with identical input data: 84,015 drawing instructions, all identical. This guarantees that renaming files, variables and functions did not change the behavior.

On top of these checks there is the author's hands-on test in the browser, which confirmed correct operation. It must be stated clearly that the simulation does not replace a test in a real browser: the project has not been verified on every browser and device, and the repository does not yet contain automated tests.

11. Known limitations

- RMS and peak are simple measurements in dBFS. Professional tools use **LUFS** (perceived loudness, EBU R128 standard) and *true peak*, which are more complex calculations.
- The analysis windows of consecutive frames overlap (each window lasts about 93 ms and a frame arrives roughly every 17 ms), so the measurements are an approximation and not those of a standardized meter.
- Frequency resolution in the bass is limited (about 10.8 Hz per bin) and is compensated by interpolation.
- The canvas has a fixed size in pixels. On narrow screens the CSS shrinks it, but the text becomes smaller as well.
- Settings are not saved between sessions.
- Microphone operation depends on browser permissions, and when the file is opened directly from disk some browsers may behave differently from a site served over HTTPS.

12. Possible future work

- **LUFS** measurement according to EBU R128, and *true peak*.
- **Beat detection** and visualizations that react to rhythm.
- A **spectrogram of the whole track**, in addition to the real-time one.

- **Recording a video clip** of the visualization with `MediaRecorder`.
- **Migration to TypeScript** with Vite, to add type checking and to demonstrate refactoring skills.
- **Automated tests** in the repository.
- A truly resizable canvas, with handling of the screen's pixel density.
- A link with the **DSP lab in Python and NumPy**, the next portfolio project: implement FFT and filters from scratch and compare the results with this application.

13. Running and publishing

To try the project, just open `index.html` with a double click. Alternatively, with Python installed, a local server can be started from the project folder with `python -m http.server 8000`, and then `http://localhost:8000` can be opened.

The next step is publishing on GitHub: installing Git, creating the repository, writing a README with a preview GIF and enabling GitHub Pages to obtain an online address to put on a résumé. The published site uses HTTPS, so the microphone will work without restrictions.

Appendix A: Mapping between original and renamed identifiers

Original name	Name in Sonoscopio	Meaning
`main.js`	`sonoscopio.js`	behavior file
`style.css`	`stile.css`	appearance file
`canvas` / `g`	`tela` / `pennello`	the canvas and its drawing context
`audioContext`	`contestoAudio`	the audio mixer
`analyser`	`analizzatore`	the node that measures the sound
`frequencyData` / `timeData`	`datiFrequenza` / `datiOnda`	spectrum and waveform
`FFT_SIZE`	`DIMENSIONE_FFT`	analysis detail
`PLOT_X`, `PLOT_Y`,	`GRAFICO_X`, `GRAFICO_Y`,	

`PLOT_W`, `PLOT_H`	`GRAFICO_LARGHEZZA`, `GRAFICO_ALTEZZA`	graph area
`setupAudio`	`preparaAudio`	building the audio chain
`getBandValue`	`valoreBanda`	value of a frequency band
`traceCurve`	`tracciaCurva`	smooth curve
`updateLevels`	`aggiornaLivelli`	RMS and peak calculation
`drawBars` / `drawCurve` / `drawSpectrogram`	`disegnaBarre` / `disegnaCurva` / `disegnaSonogramma`	the three modes
`draw`	`disegna`	animation loop
`startMicrophone` / `stopMicrophone`	`avviaMicrofono` / `fermaMicrofono`	microphone handling
`loadAudioFile`	`caricaFileAudio`	loading the file
`bars` / `curve` / `spectrogram`	`barre` / `curva` / `sonogramma`	values of the mode menu

Names that belong to the browser itself (such as `getContext`, `createAnalyser` or `getByteFrequencyData`) cannot be changed and stay the same.

Appendix B: Glossary

- **Sample**: a single numeric value of the sound wave at one instant.
- **Sampling rate**: how many samples per second (44,100 or 48,000 in common audio).
- **FFT**: an algorithm that breaks a sound down into its frequencies.
- **Bin**: a slice of the spectrum produced by the FFT.
- **Octave**: an interval in which the frequency doubles.
- **dBFS**: decibels relative to the maximum possible level in digital audio (0 dB).
- **RMS**: the average level of a signal, computed as the root of the mean of the squares.
- **Peak**: the highest sample in absolute value.
- **Spectrogram**: a graph of time, frequency and intensity (color); called *sonogramma* in

the project.

- **Canvas**: an HTML element on which one draws with code.
- **Frame**: a single image of the animation, about 60 per second.
- **Exponential moving average**: an average in which recent values weigh more than old ones.
- **Feedback (Larsen effect)**: a howl caused by sound from the speakers re-entering the microphone.